

DevGPT: Studying Developer-ChatGPT Conversations

Tao Xiao
Nara Institute of Science and Technology
Japan
tao.xiao.ts2@is.naist.jp

Hideaki Hata
Shinshu University
Japan
hata@shinshu-u.ac.jp

Christoph Treude
University of Melbourne
Australia
christoph.treude@unimelb.edu.au

Kenichi Matsumoto
Nara Institute of Science and Technology
Japan
matumoto@is.naist.jp

ABSTRACT

This paper introduces DevGPT, a dataset curated to explore how software developers interact with ChatGPT, a prominent large language model (LLM). The dataset encompasses 29,778 prompts and responses from ChatGPT, including 19,106 code snippets, and is linked to corresponding software development artifacts such as source code, commits, issues, pull requests, discussions, and Hacker News threads. This comprehensive dataset is derived from shared ChatGPT conversations collected from GitHub and Hacker News, providing a rich resource for understanding the dynamics of developer interactions with ChatGPT, the nature of their inquiries, and the impact of these interactions on their work. DevGPT enables the study of developer queries, the effectiveness of ChatGPT in code generation and problem solving, and the broader implications of AI-assisted programming. By providing this dataset, the paper paves the way for novel research avenues in software engineering, particularly in understanding and improving the use of LLMs like ChatGPT by developers.

CCS CONCEPTS

• Information systems → Data mining.

KEYWORDS

ChatGPT, LLM, Generative AI, dataset

ACM Reference Format:

Tao Xiao, Christoph Treude, Hideaki Hata, and Kenichi Matsumoto. 2024. DevGPT: Studying Developer-ChatGPT Conversations. In *21st International Conference on Mining Software Repositories (MSR '24)*, April 15–16, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3643991.3648400>

1 HIGH-LEVEL OVERVIEW

The emergence of large language models (LLMs) such as ChatGPT has disrupted the landscape of software development. Many studies

are investigating the quality of responses generated by ChatGPT, the efficacy of various prompting techniques, and its comparative performance in programming contests, to name a few examples. Yet, we know very little about how ChatGPT is actually used by software developers. What questions do developers present to ChatGPT? What are the dynamics of these interactions? What is the backdrop against which these conversations are held, and how do the conversations feedback into the artifacts of their work? To close this gap, we introduce DevGPT, a curated dataset which encompasses 29,778 prompts and ChatGPT's responses including 19,106 code snippets, coupled with the corresponding software development artifacts—ranging from source code, commits, issues, pull requests, to discussions and Hacker News threads—to enable the analysis of the context and implications of these developer interactions with ChatGPT.

To create DevGPT, we leveraged a feature introduced by OpenAI in late May 2023, which allows users to share their interactions with ChatGPT through dedicated links.¹ We collected all such links shared on GitHub and Hacker News at nine specific points from July to October. If users chose to delete or deactivate their shared conversations in the intervening periods, we ensured data consistency by accessing the original shared link across all these snapshots.

Table 1 provides an overview of the snapshot 20231012. Comprising 4,733 shared ChatGPT links sourced from 3,559 GitHub or Hacker News references, the dataset contains a total of 29,778 prompts/answers. This includes 19,106 code snippets, with Python (6,084), JavaScript (4,802), and Bash (4,332) as the top three programming languages. 940 of these links are referenced across multiple sources, resulting in a unique count of 3,794 individual ChatGPT shared links within DevGPT.

Figure 1 shows an instance of a ChatGPT conversation from the dataset, together with the pull request it was related to and how the code was updated after the ChatGPT conversation.

2 INTERNAL STRUCTURE

The dataset consists of a collection of JSON files collected from the six sources detailed in Table 1. For each source, we provide distinct metadata in the JSON file to enable source-specific analysis. Apart from the source-specific metadata, every JSON contains a consistent attribute: a list of shared ChatGPT links. Each shared link includes the URL to the ChatGPT conversation, the associated HTTP response status codes, the access date of the URL, and

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR '24, April 15–16, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0587-8/24/04

<https://doi.org/10.1145/3643991.3648400>

¹<https://help.openai.com/en/articles/7925741-chatgpt-shared-links-faq>

Table 1: Summary Statistics of the snapshot 20231012

Sources	#	Mentioned in	Shared ChatGPT Links			ChatGPT Conversations	
			# Shared Links	# Accessible Links	# Conversations with Code	# Prompts	# Code Snippets
GitHub Code File	1,843	Code	2,708	2,540	1,184	22,799	14,132
GitHub Commit	694	Message	694	692	674	1,922	1,828
GitHub Issue	507	Comment	404	382	215	1,212	821
		Description	228	212	141	1,103	841
		Title	4	4	4	50	77
GitHub Pull Request	267	Description	94	93	59	529	384
		Review Thread	109	102	66	201	166
		Comment	98	91	54	430	425
Hacker News	187	Comment	267	234	44	849	127
		Attached URL	42	37	2	376	54
		Story	15	12	4	48	63
GitHub Discussion	61	Comment	40	34	17	138	76
		Description	21	20	12	93	87
		Reply	9	7	5	28	25

Figure 1: Example of a ChatGPT conversation in the context of a GitHub pull request

The figure shows a GitHub pull request interface. At the top, the pull request title is "FlatAHB Dump: Don't use new GUID for otherwise unchanged lines" with a status of "Merged". Below the title, the author "FlatAHB" is listed. The pull request is reviewed by "Reviewer" 3 weeks ago. The review comment discusses a C# pattern for deep copies and mentions a ChatGPT conversation. The ChatGPT conversation is shown in a separate window, with a prompt asking for a deep copy in Python and an answer providing a code snippet. The answer includes a code snippet for a deep copy function. The conversation is annotated with "ChatGPT shared conversation" and "Modified commit based on the review".

the content within the HTML response. Additionally, each conversation contains a list of prompts/answers, inclusive of any code snippets. We provide details including the date of the conversation, the count of prompts/answers, their token information, and the model version involved in the chat. Attributes detailing where the conversation was referenced are also included—such as the referencing URL, the nature of the mention (e.g., a comment), the individual who mentioned it, and the context in which it was cited. A comprehensive breakdown of the data structure is available at <https://github.com/NAIST-SE/DevGPT>. Additionally, we provide a CSV file cataloging all shared ChatGPT links gathered from GitHub and Hacker News.

3 HOW TO ACCESS

The DevGPT dataset is available for download on Zenodo, see Section 6. It is formatted in JSON, making it easily parsable with any standard JSON library. Additionally, we include the HTTP response, which can be analyzed using any HTML parser. The dataset also categorizes code snippets by type, enabling researchers to use corresponding compilers for execution. No credentials are needed to access the dataset.

4 POTENTIAL RESEARCH QUESTIONS

The following provides a sample list of research questions that can be answered with the DevGPT dataset:

- (1) What types of issues (bugs, feature requests, theoretical questions, etc.) do developers most commonly present to ChatGPT?
- (2) Can we identify patterns in the prompts developers use when interacting with ChatGPT, and do these patterns correlate with the success of issue resolution?
- (3) What is the typical structure of conversations between developers and ChatGPT? How many turns does it take on average to reach a conclusion?
- (4) In instances where developers have incorporated the code provided by ChatGPT into their projects, to what extent do they modify this code prior to use, and what are the common types of modifications made?
- (5) How does the code generated by ChatGPT for a given query compare to code that could be found for the same query on the internet (e.g., on Stack Overflow)?
- (6) What types of quality issues (for example, as identified by linters) are common in the code generated by ChatGPT?
- (7) How accurately can we predict the length of a conversation with ChatGPT based on the initial prompt and context provided?
- (8) Can we reliably predict whether a developer's issue will be resolved based on the initial conversation with ChatGPT?
- (9) If developers were to rerun their prompts with ChatGPT now and/or with different settings, would they obtain the same results?

5 RELATED WORK

To situate the DevGPT dataset in the existing literature, in this section, we discuss existing research on link sharing and large language models (LLMs) in the field of software engineering.

5.1 Link Sharing

Link sharing, a prevalent method of knowledge sharing, is extensively adopted within developer communities, including Q&A sites, GitHub, and code reviews. Gómez et al. [10] found that a considerable number of links on Stack Overflow were used to share knowledge about software development innovations, such as libraries and tools. Ye et al. [38] examined the structural and dynamic aspects of the knowledge network on Stack Overflow, noting that developers use links for various purposes, predominantly for referencing information to solve problems. Hata et al. [12] noted that over 80% of repositories feature at least one link in source code comments. Xiao et al. [35] expanded this research to include the role of links in commit messages, observing that inaccessible and patch links were most common. The practice of link sharing was also studied in the context of code review. Zampetti et al. [40] explored the extent and purpose of external online resource references in pull requests, finding that developers often consult external resources to gain knowledge or resolve specific issues. Wang et al. [30] employed a mixed-method approach to underscore the importance of shared links in review discussions, highlighting their role in satisfying the information needs of patch authors and review teams.

5.2 LLMs for SE

Since the introduction of the Transformer architecture in 2017 [29], LLMs have become increasingly significant in Software Engineering (SE). Hou et al. [13] conducted a systematic review of 229 research articles from 2017 to 2023, revealing the widespread use of LLMs in addressing software development problems. Prominent models in this area include GPT-2/GPT-3/GPT-3.5 [7, 17, 19, 20, 23, 31, 39], GPT-4 [3, 9, 14, 20], and the BERT series [16, 41], demonstrating effectiveness in code generation, completion, and summarization.

Code completion, integral to Integrated Development Environments (IDEs) and code editors, has been enhanced by tools like Codex [5, 6, 18, 25], the BERT series [15], GitHub Copilot [6, 18, 26], CodeParrot [18, 37], and the GPT series [24, 37]. Conversely, code summarization technologies like Codex [1, 2, 8], CodeBERT [4, 8, 11], and T5 [21, 22] focus on generating natural language descriptions from source code to facilitate maintenance, search, and classification.

In software maintenance, nearly a quarter of the studies reviewed by Hou et al. [13] address program repair, code review, and debugging. In program repair, Codex [32, 33] and ChatGPT [34] have shown strong performance. For code review, LLMs like BERT [27] and ChatGPT [28] are effective in detecting issues and suggesting optimizations. Additionally, Copilot for PRs powered pull requests need less review time and have a higher likelihood of being merged [36].

Despite these advances, there is limited research on how software developers interact with LLMs. The DevGPT dataset addresses this gap, offering a valuable resource for in-depth analysis of these interactions. This dataset can enable the research community to understand and improve the ways developers use LLMs in their work, marking a step forward in the practical application of AI in software development.

6 LINKS

<https://github.com/NAIST-SE/DevGPT> and <https://doi.org/10.5281/zenodo.10086809>

REFERENCES

- [1] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl T Barr. 2023. Improving Few-Shot Prompts with Relevant Static Analysis Products. *arXiv preprint arXiv:2304.06815* (2023).
- [2] Shushan Arakelyan, Rocktim Jyoti Das, Yi Mao, and Xiang Ren. 2023. Exploring Distributional Shifts in Large Language Models for Code Analysis. *arXiv preprint arXiv:2303.09128* (2023).
- [3] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code. *arXiv preprint arXiv:2206.01335* (2022).
- [4] Fuxiang Chen, Fatemeh H Fard, David Lo, and Timofey Bryksin. 2022. On the transferability of pre-trained language models for low-resource programming languages. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*. 401–412.
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [6] Jean-Baptiste Döderlein, Mathieu Acher, Djamel Eddine Khelladi, and Benoit Combemale. 2022. Piloting Copilot and Codex: Hot Temperature, Cold Prompts, or Black Magic? *arXiv preprint arXiv:2210.14699* (2022).
- [7] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration Code Generation via ChatGPT. *arXiv preprint arXiv:2304.07590* (2023).
- [8] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, and Michael R Lyu. 2023. Constructing Effective In-Context Demonstration for Code Intelligence Tasks: An Empirical Study. *arXiv preprint arXiv:2304.07575* (2023).

- [9] Henry Gilbert, Michael Sandborn, Douglas C Schmidt, Jesse Spencer-Smith, and Jules White. 2023. Semantic Compression With Large Language Models. *arXiv preprint arXiv:2304.12512* (2023).
- [10] Carlos Gómez, Brendan Cleary, and Leif Singer. 2013. A study of innovation diffusion through link sharing on stack overflow. In *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 81–84.
- [11] Jian Gu, Pasquale Salza, and Harald C Gall. 2022. Assemble foundation models for automatic code summarization. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 935–946.
- [12] Hideaki Hata, Christoph Treude, Raula Gaikovina Kula, and Takashi Ishio. 2019. 9.6 million links in source code comments: Purpose, evolution, and decay. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 1211–1221.
- [13] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *arXiv preprint arXiv:2308.10620* (2023).
- [14] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *arXiv preprint arXiv:2306.02907* (2023).
- [15] Junaed Younus Khan and Gias Uddin. 2022. Automatic detection and analysis of technical debts in peer-review documentation of r packages. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 765–776.
- [16] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*. PMLR, 18319–18345.
- [17] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023. Enabling Programming Thinking in Large Language Models Toward Code Generation. *arXiv preprint arXiv:2305.06599* (2023).
- [18] Zongjie Li, Chaozheng Wang, Zhibo Liu, Haoxuan Wang, Dong Chen, Shuai Wang, and Cuiyun Gao. 2023. Cctest: Testing and repairing code completion systems. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1238–1250.
- [19] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. 2023. Improving ChatGPT Prompt for Code Generation. *arXiv preprint arXiv:2305.08360* (2023).
- [20] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *arXiv preprint arXiv:2305.01210* (2023).
- [21] Antonio Mastropaolo, Luca Pascarella, and Gabriele Bavota. 2022. Using deep learning to generate complete log statements. In *Proceedings of the 44th International Conference on Software Engineering*. 2279–2290.
- [22] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 336–347.
- [23] Nathalia Nascimento, Paulo Alencar, and Donald Cowan. 2023. Comparing Software Developers with ChatGPT: An Empirical Investigation. *arXiv preprint arXiv:2305.11837* (2023).
- [24] Marcel Ochs, Krishna Narasimhan, and Mira Mezini. 2023. Evaluating and improving transformers pre-trained on ASTs for Code Completion. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 834–844.
- [25] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356.
- [26] Rohith Pudari and Neil A Ernst. 2023. From Copilot to Pilot: Towards AI Supported Software Development. *arXiv preprint arXiv:2303.04142* (2023).
- [27] Oussama Ben Sghaier and Houari Sahraoui. 2023. A Multi-Step Learning Approach to Assist Code Review. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 450–460.
- [28] Giriprasad Sridhara, Sourav Mazumdar, et al. 2023. ChatGPT: A Study on its Utility for Ubiquitous Software Engineering Tasks. *arXiv preprint arXiv:2305.16837* (2023).
- [29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [30] Dong Wang, Tao Xiao, Patanamorn Thongtanunam, Raula Gaikovina Kula, and Kenichi Matsumoto. 2021. Understanding shared links and their intentions to meet information needs in modern code review: A case study of the OpenStack and Qt projects. *Empirical Software Engineering* 26 (2021), 1–32.
- [31] Jian Wang, Shangqing Liu, Xiaofei Xie, and Yi Li. 2023. Evaluating AIGC Detectors on Code Content. *arXiv preprint arXiv:2304.05193* (2023).
- [32] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How Effective Are Neural Networks for Fixing Security Vulnerabilities. *arXiv preprint arXiv:2305.18607* (2023).
- [33] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023)*. Association for Computing Machinery.
- [34] Chunqiu Steven Xia and Lingming Zhang. 2023. Conversational automated program repair. *arXiv preprint arXiv:2301.13246* (2023).
- [35] Tao Xiao, Sebastian Balthes, Hideaki Hata, Christoph Treude, Raula Gaikovina Kula, Takashi Ishio, and Kenichi Matsumoto. 2023. 18 million links in commit messages: purpose, evolution, and decay. *Empirical Software Engineering* 28, 4 (2023), 91.
- [36] Tao Xiao, Hideaki Hata, Christoph Treude, and Kenichi Matsumoto. 2024. Generative AI for Pull Request Descriptions: Adoption, Impact, and Developer Interventions. In *Proceedings of the ACM on Software Engineering (PACMSE)*.
- [37] Frank F Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 1–10.
- [38] Deheng Ye, Zhenchang Xing, and Nachiket Kapre. 2017. The structure and dynamics of knowledge network in domain-specific q&a sites: a case study of stack overflow. *Empirical Software Engineering* 22, 1 (2017), 375–406.
- [39] Burak Yetiştir, İşık Özsoy, Miray Ayerdem, and Eray Tüzün. 2023. Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. *arXiv preprint arXiv:2304.10778* (2023).
- [40] Fiorella Zampetti, Luca Ponzanelli, Gabriele Bavota, Andrea Mocchi, Massimiliano Di Penta, and Michele Lanza. 2017. How developers document pull requests with external references. In *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*. IEEE, 23–33.
- [41] Zhengran Zeng, Hanzhuo Tan, Haotian Zhang, Jing Li, Yuqun Zhang, and Lingming Zhang. 2022. An extensive study on pre-trained models for program understanding and generation. In *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*. 39–51.